GUIA DE AULA – PROJETO PAINEL DE DESPESAS

Objetivo: orientar os alunos na construção da estrutura HTML de um painel de controle de despesas. O projeto servirá como base para as próximas etapas com CSS e JavaScript/TypeScript.

1. O que os alunos vão construir?

Os alunos desenvolverão uma aplicação chamada **Painel de Despesas**. O usuário poderá informar uma despesa, seu valor e sua categoria. Posteriormente, o sistema poderá listar as despesas, calcular o valor total e mostrar os totais por categoria.

Funcionalidades previstas

Funcionalidade	Descrição
Cadastrar despesa	Informar título, valor e categoria.
Listar despesas	Mostrar dinamicamente as despesas cadastradas.
Calcular total	Somar todos os valores cadastrados.
Agrupar categorias	Mostrar quanto foi gasto em cada categoria.
Validar formulário	Impedir o cadastro de dados inválidos.
Excluir despesa	Remover uma despesa da lista (etapa posterior).

2. Estrutura básica do documento HTML

Todo documento HTML começa informando ao navegador qual tipo de documento está sendo utilizado. Também definimos o idioma da página e as configurações básicas dentro da seção **head**.

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="/src/style.css" />
  <title>Painel de Despesas</title>
</head>
```

Explicação dos principais elementos

• **<!DOCTYPE html>**: informa ao navegador que estamos utilizando HTML5.

• **lang="pt-BR"**: define o idioma principal da página como português do Brasil.

• **<meta charset="UTF-8">**: permite utilizar corretamente acentos e caracteres especiais.

• **viewport**: ajuda a página a se adaptar melhor a diferentes tamanhos de tela.

• **<link rel="stylesheet">**: conecta o arquivo CSS ao HTML.

• **<title>**: define o texto exibido na aba do navegador.

Atenção: erro comum

O atributo correto é **stylesheet**. Se o aluno escrever **styleesheet**, o arquivo CSS não será carregado corretamente.

3. Estrutura principal da aplicação

Dentro do **body** está toda a interface visível para o usuário. O projeto foi dividido em duas grandes áreas: uma barra lateral e o conteúdo principal.

```
<body>
  <div id="app">

    <aside id="sidebar">
      <!-- Área de resumo -->
    </aside>

    <main id="conteudo">
      <!-- Conteúdo principal -->
    </main>

  </div>
</body>
```

Por que dividir a página dessa forma?

A organização em áreas facilita a criação do layout com CSS. A **aside** normalmente representa um conteúdo complementar, enquanto o **main** representa o conteúdo principal da página.

4. Barra lateral: resumo financeiro

```
<aside id="sidebar">

  <h2 id="sidebarTitulo" class="sidebar__titulo">
    Resumo
  </h2>

  <p class="sidebar__rotulo-total">
    Soma total
  </p>

  <p id="somaTotalDespesas" class="sidebar__valor-total">
    R$ 0,00
  </p>

  <hr class="sidebar__divisor" />

  <p class="sidebar__rotulo-categorias">
    Totais por categoria
  </p>

  <ul
    id="totaisPorCategoria"
    class="sidebar__lista-categorias"
  ></ul>

</aside>
```

O elemento **somaTotalDespesas** será atualizado posteriormente pelo JavaScript/TypeScript. A lista **totaisPorCategoria** começa vazia porque seus itens serão criados dinamicamente.

5. Card para cadastrar uma nova despesa

Utilizamos uma **section** para representar uma parte específica da aplicação. A classe **card** pode ser reutilizada para aplicar o mesmo estilo visual em diferentes áreas.

```
<section id="cardNovaDespesa" class="card">

  <h2 class="card__titulo">
    Nova Despesa
  </h2>

  <form
    id="formNovaDespesa"
    class="form-nova-despesa"
    novalidate
  >
    <!-- Campos do formulário -->
  </form>

</section>
```

O formulário

O elemento **form** agrupa os campos utilizados para enviar ou processar informações. O atributo **id="formNovaDespesa"** permite localizar o formulário posteriormente pelo JavaScript.

O atributo **novalidate** desativa a validação padrão do navegador. Isso é útil quando os alunos irão criar sua própria validação com JavaScript/TypeScript.

6. Campo de título

```
<div class="campo">

  <label for="tituloDespesa">
    Título
  </label>

  <input
    type="text"
    id="tituloDespesa"
    name="titulo"
    placeholder="Ex.: Mercado do mês"
  />

</div>
```

label identifica o campo para o usuário. O atributo **for** deve apontar para o **id** do input correspondente. O **placeholder** apresenta um exemplo dentro do campo.

7. Campo de valor

```
<input
  type="number"
  id="valorDespesa"
  name="valor"
  placeholder="Ex.: 150,00"
  step="0.01"
  min="0.01"
/>
```

type="number" indica que esperamos um número. **step="0.01"** permite valores decimais. **min="0.01"** define o menor valor permitido.

8. Campo de categoria

```
<select id="categoriaDespesa" name="categoria">

  <option value="" disabled selected>
    Selecione uma categoria
  </option>

  <option value="Alimentos">Alimentos</option>
  <option value="Transporte">Transporte</option>
  <option value="Lazer">Lazer</option>
  <option value="Saude">Saúde</option>
  <option value="Cartao">Cartão de Crédito</option>
  <option value="Outros">Outros</option>

</select>
```

O elemento **select** cria uma lista de opções. Cada **option** representa uma categoria. A primeira opção funciona como instrução e possui valor vazio.

9. Botão para adicionar

```
<button
  type="submit"
  id="botaoAdicionarDespesa"
  class="botao-adicionar"
>
  Adicionar despesa
</button>
```

Como o botão está dentro do formulário e possui **type="submit"**, o evento de envio do formulário poderá ser capturado no JavaScript/TypeScript.

10. Área para mensagens de erro

```
<p
  id="mensagemErroValidacao"
  class="mensagem-erro"
  hidden
></p>
```

O atributo **hidden** faz com que a mensagem comece escondida. Quando houver um erro de validação, o JavaScript poderá inserir uma mensagem e remover esse estado.

11. Área de listagem das despesas

```
<section id="cardListaDespesas" class="card">

  <h2 class="card__titulo">
    Despesas
  </h2>

  <ul
    id="listaDespesas"
    class="lista-despesas"
  ></ul>

  <p
    id="mensagemListaVazia"
    class="mensagem-vazia"
  >
    Nenhuma despesa cadastrada ainda.
  </p>

</section>
```

A lista começa vazia. Quando uma despesa for cadastrada, o JavaScript/TypeScript poderá criar um elemento **li** e adicioná-lo dentro de **listaDespesas**.

12. Conexão com TypeScript

```
<script
  type="module"
  src="/src/main.ts"
></script>
```

Esse código carrega o arquivo principal do projeto. Como o atributo **type="module"** é utilizado, o projeto pode trabalhar com módulos e importações.

Fluxo esperado da aplicação

1. O usuário preenche título, valor e categoria.
2. O usuário clica em Adicionar despesa.
3. O sistema valida os dados.
4. A despesa é adicionada a uma estrutura de dados.
5. A interface é atualizada.
6. O valor total é recalculado.
7. Os totais por categoria são atualizados.

13. IDs importantes para o JavaScript/TypeScript

ID	Responsabilidade
formNovaDespesa	Capturar o envio do formulário.
tituloDespesa	Ler o título informado.
valorDespesa	Ler o valor informado.
categoriaDespesa	Ler a categoria selecionada.
listaDespesas	Exibir as despesas cadastradas.
somaTotalDespesas	Mostrar a soma total.
totaisPorCategoria	Mostrar os valores agrupados por categoria.
mensagemErroValidacao	Mostrar mensagens de erro.
mensagemListaVazia	Informar que ainda não existem despesas.

14. Desafio para os alunos

Etapa 1 – HTML: criar toda a estrutura apresentada neste guia.

Etapa 2 – CSS: transformar a estrutura em um painel visualmente organizado e responsivo.

Etapa 3 – JavaScript/TypeScript: implementar o cadastro e a validação das despesas.

Etapa 4 – Atualização da interface: listar as despesas e calcular os totais.

Desafio extra: permitir excluir uma despesa e atualizar automaticamente todos os valores.

Regra para o projeto: o aluno deve entender o código que desenvolveu. A apresentação do funcionamento e a explicação da lógica fazem parte da atividade.